# 6. TensorFlow

## Introduction

TensorFlow is an open-source Deep Learning framework developed by Google for building Artificial Intelligence (AI) and Machine Learning applications. It performs numerical computations using multidimensional arrays called **tensors**, making it highly efficient for developing neural networks and deep learning models. TensorFlow supports execution on CPUs, GPUs, and TPUs, enabling fast training of large-scale models. It is widely used in applications such as image recognition, speech recognition, natural language processing, recommendation systems, and predictive analytics. TensorFlow also provides high-level APIs like Keras, making model development easier for beginners while still offering advanced features for researchers and professionals.

## Import Library

```
import tensorflow as tf
```

## Common Functions

1. **tf.constant()** – Creates a constant tensor.
2. **tf.Variable()** – Creates a trainable variable.
3. **tf.random.normal()** – Generates random values from a normal distribution.
4. **tf.random.uniform()** – Generates uniformly distributed random values.
5. **tf.zeros()** – Creates a tensor filled with zeros.
6. **tf.ones()** – Creates a tensor filled with ones.
7. **tf.fill()** – Creates a tensor filled with a specified value.
8. **tf.range()** – Generates a sequence of numbers.
9. **tf.reshape()** – Changes the shape of a tensor.
10. **tf.cast()** – Changes the data type of a tensor.
11. **tf.add()** – Adds two tensors.
12. **tf.subtract()** – Subtracts one tensor from another.
13. **tf.multiply()** – Multiplies two tensors.
14. **tf.divide()** – Divides one tensor by another.
15. **tf.matmul()** – Performs matrix multiplication.
16. **tf.reduce_sum()** – Computes the sum of tensor elements.
17. **tf.reduce_mean()** – Computes the mean of tensor elements.
18. **tf.reduce_max()** – Finds the maximum value.
19. **tf.reduce_min()** – Finds the minimum value.
20. **tf.square()** – Computes the square of each element.
21. **tf.sqrt()** – Computes the square root.
22. **tf.exp()** – Computes the exponential value.
23. **tf.math.log()** – Computes the natural logarithm.
24. **tf.argmax()** – Returns the index of the largest value.
25. **tf.argmin()** – Returns the index of the smallest value.

```
import tensorflow as tf

# 1. tf.constant()
a = tf.constant([1,2,3])
print("1.", a)

# 2. tf.Variable()
b = tf.Variable([4,5,6])
print("2.", b)

# 3. tf.random.normal()
print("3.\n", tf.random.normal((2,2)))

# 4. tf.random.uniform()
print("4.\n", tf.random.uniform((2,2)))

# 5. tf.zeros()
print("5.\n", tf.zeros((2,3)))

# 6. tf.ones()
print("6.\n", tf.ones((2,3)))
```

```
# 7. tf.fill()
print("7.\n", tf.fill((2,2),7))

# 8. tf.range()
print("8.", tf.range(1,11))

# 9. tf.reshape()
x = tf.range(1,13)
print("9.\n", tf.reshape(x,(3,4)))

# 10. tf.cast()
print("10.", tf.cast([1.2,2.8,3.5],dtype=tf.int32))

# 11. tf.add()
print("11.", tf.add(10,20))

# 12. tf.subtract()
print("12.", tf.subtract(20,5))

# 13. tf.multiply()
print("13.", tf.multiply(5,6))

# 14. tf.divide()
print("14.", tf.divide(20,4))

# 15. tf.matmul()
m1 = tf.constant([[1,2],[3,4]])
m2 = tf.constant([[5,6],[7,8]])
print("15.\n", tf.matmul(m1,m2))

# 16. tf.reduce_sum()
print("16.", tf.reduce_sum(a))

# 17. tf.reduce_mean()
print("17.", tf.reduce_mean(tf.cast(a,tf.float32)))

# 18. tf.reduce_max()
print("18.", tf.reduce_max(a))

# 19. tf.reduce_min()
print("19.", tf.reduce_min(a))

# 20. tf.square()
print("20.", tf.square(a))

# 21. tf.sqrt()
print("21.", tf.sqrt(tf.constant([4.0,9.0,16.0])))

# 22. tf.exp()
print("22.", tf.exp(tf.constant([1.0,2.0])))

# 23. tf.math.log()
print("23.", tf.math.log(tf.constant([1.0,2.0,10.0])))

# 24. tf.argmax()
print("24.", tf.argmax([5,8,2,9,4]))

# 25. tf.argmin()
print("25.", tf.argmin([5,8,2,9,4]))
```

```
1. tf.Tensor([1 2 3], shape=(3,), dtype=int32)
2. <tf.Variable 'Variable:0' shape=(3,) dtype=int32, numpy=array([4, 5, 6], dtype=int32)>
3.
 tf.Tensor(
[[ 0.58508426  0.46322364]
 [ 0.5006535  -0.2316481 ]], shape=(2, 2), dtype=float32)
4.
 tf.Tensor(
[[0.61665857 0.29480696]
 [0.5604093  0.12421751]], shape=(2, 2), dtype=float32)
5.
 tf.Tensor(
[[0. 0. 0.]
 [0. 0. 0.]], shape=(2, 3), dtype=float32)
6.
 tf.Tensor(
[[1. 1. 1.]
 [1. 1. 1.]], shape=(2, 3), dtype=float32)
7.
 tf.Tensor(
[[7 7]
 [7 7]], shape=(2, 2), dtype=int32)
8. tf.Tensor([ 1  2  3  4  5  6  7  8  9 10], shape=(10,), dtype=int32)
9.
 tf.Tensor(
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]], shape=(3, 4), dtype=int32)
10. tf.Tensor([1 2 3], shape=(3,), dtype=int32)
11. tf.Tensor(30, shape=(), dtype=int32)
12. tf.Tensor(15, shape=(), dtype=int32)
13. tf.Tensor(30, shape=(), dtype=int32)
14. tf.Tensor(5.0, shape=(), dtype=float64)
15.
 tf.Tensor(
[[19 22]
 [43 50]], shape=(2, 2), dtype=int32)
16. tf.Tensor(6, shape=(), dtype=int32)
17. tf.Tensor(2.0, shape=(), dtype=float32)
18. tf.Tensor(3, shape=(), dtype=int32)
19. tf.Tensor(1, shape=(), dtype=int32)
20. tf.Tensor([1 4 9], shape=(3,), dtype=int32)
21. tf.Tensor([2. 3. 4.], shape=(3,), dtype=float32)
22. tf.Tensor([2.7182817 7.389056 ], shape=(2,), dtype=float32)
23. tf.Tensor([0.        0.6931472 2.3025851], shape=(3,), dtype=float32)
24. tf.Tensor(3, shape=(), dtype=int64)
25. tf.Tensor(2, shape=(), dtype=int64)
```

Start coding or generate with AI.